

Array_2D.java

```
1 // =====***** Arrays_2D
  *****===== //
2
3 import java.util.*;
4 public class Array_2D {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7         int matrix [][] = new int [3][3];
8         int n = 3;           // int n = matrix.length;
9         int m = 3;           // int m = matrix[0].length;
10
11         // Input
12         for(int i=0; i<n; i++){
13             for(int j=0; j<m; j++){
14                 matrix [i][j] = sc.nextInt();
15             }
16         }
17
18         // Output
19         for(int i=0; i<n; i++){
20             for(int j=0; j<m; j++){
21                 System.out.print(matrix [i][j] + " ");
22             }
23             System.out.println();
24         }
25
26         Search(matrix, 8);
27     }
28     public static boolean Search(int matrix [][], int key){
29         for(int i=0; i<matrix.length; i++){
30             for(int j=0; j<matrix[0].length; j++){
31                 if(matrix[i][j] == key){
32                     System.out.println("Index at found (" +i+ "," +j+)");
33                     return true;
34                 }
35             }
36         }
37         System.out.println("Index not found ");
38         return false;
39     }
40 }
41
42 // =====> Spiral Matrix <=====
43 class Sprial_Matrix{
44     public static void sprial_Matrix(int matrix[][]){
45         int startRow = 0;
46         int startCol = 0;
47         int endRow = matrix.length-1;
48         int endCol = matrix[0].length-1;
```

```

49
50     while(startRow <= endRow && startCol <= endCol){
51         // Top
52         for(int j=startCol; j<=endCol; j++){
53             System.out.print(matrix[startRow][j] + " ");
54         }
55
56         // Right
57         for(int i=startRow+1; i<=endRow; i++){
58             System.out.print(matrix[i][endCol] + " ");
59         }
60
61         // Bottom
62         for(int j=endCol-1; j>=startCol; j--){
63             if(startRow == endRow){
64                 break;
65             }
66             System.out.print(matrix[endRow][j] + " ");
67         }
68
69         // Left
70         for(int i=endRow-1; i>=startRow+1; i--){
71             if(startCol == endCol){
72                 break;
73             }
74             System.out.print(matrix[i][startCol] + " ");
75         }
76
77         startCol++;
78         startRow++;
79         endCol--;
80         endRow--;
81     }
82     System.out.println();
83 }
84 public static void main(String[] args) {
85     int matrix [][] = {{1, 2, 3, 4},
86                       {5, 6, 7, 8},
87                       {9, 10, 11, 12},
88                       {13, 14, 15, 16}};
89
90     sprial_Matrix(matrix);
91 }
92 }
93
94
95 // =====> Diagonal Sum Matrix <=====
96 public class Diagonal_Sum {
97     public static int Diagonal(int matrix [][]){
98         int sum = 0;
99         /* =====> First Method {Tc ---> O(n^2)}

```

```

100     for(int i=0; i<matrix.length; i++){
101         for(int j=0; j<matrix[0].length; j++){
102             if(i == j){
103                 sum+=matrix[i][j];
104             }
105             else if(i+j == matrix.length-1){
106                 sum+=matrix[i][j];
107             }
108         }
109     }
110     return sum;
111 */
112
113 /* =====> Second Method {Tc ---> O(n)}
114 for(int i=0; i<matrix.length; i++){
115     //Primary Diagonal
116     sum+= matrix[i][i];
117     // Secondry Diagonal
118     if(i != matrix.length-1-i){
119         sum+= matrix[i][matrix.length-1-i];
120     }
121 }
122 return sum;
123 */
124 }
125 public static void main(String[] args) {
126     int matrix [][] = {{1, 2, 3, 4},
127                        {5, 6, 7, 8},
128                        {9, 10, 11, 12},
129                        {13, 14, 15, 16}};
130
131     int result = Diagonal(matrix);
132     System.out.println(result + " ");
133 }
134 }
135
136 // =====> Search in Sorted Matrix Code <=====
137
138 class Search_in_Sorted_Matrix{
139     public static boolean staircasesearch(int[][] matrix, int key){
140
141         // Method - 1 (If we using Left Bottom technique)
142         /*
143         int row = 0;
144         int col = matrix[0].length-1;
145         while(row < matrix.length && col >= 0){
146             if(matrix[row][col] == key){
147                 System.out.println("Key found at : (" + row + "," + col + ")");
148                 return true;
149             }else if(key < matrix[row][col]){
150                 col--;

```

```
151         }else{
152             row++;
153         }
154     }
155     System.out.println("key not found!");
156     return false;
157     */
158
159     // method - 2                                (If we using Top Right technique)
160     /*
161     int row = matrix.length-1;
162     int col = 0;
163     while(row>=0 && col<matrix[0].length){
164         if(key == matrix[row][col]){
165             System.out.println("Key found at : (" + row + "," + col + ")");
166             return true;
167         }else if(key<matrix[row][col]){
168             row--;
169         }else{
170             col++;
171         }
172     }
173     System.out.println("Key not found!");
174     return false;
175     */
176 }
177 public static void main(String[] args) {
178     int [][] matrix = {{10, 20, 30, 40},
179                        {15, 25, 35, 45},
180                        {27, 29, 37, 48},
181                        {32, 33, 39, 50}};
182
183     int key = 33;
184     staircasesearch(matrix, key);
185 }
186 }
187
188
189
190 // =====***** Arrays_2D Complete *****=====
191 //
```